📋 Что это такое

`--lua-desync` — это **главный механизм** desync в nfqws2. Он вызывает Lua функцию **для каждого пакета**, проходящего через профиль.

Синтаксис:

```
--lua-desync=<функция>[:параметр1=значение1[:параметр2=значение2]]
```

🔧 Как это работает

```
Пакет → профиль → --lua-desync → Lua функция → действие
```

1. Пакет проходит через фильтры профиля (`--filter-tcp` , `--filter-l7` , `--hostlist`)
2. Если пакет соответствует фильтрам → вызывается Lua функция
3. Функция выполняет действие (отправляет fake, разбивает пакет, модифицирует и т.д.)
4. Функция возвращает вердикт (`VERDICT_PASS` , `VERDICT_DROP` , `VERDICT_MODIFY`)

📦 Доступные функции из `zapret-antidpi.lua`

Базовые:

Функция	Описание
`drop`	Отбросить пакет
`send`	Отправить пакет как есть (с возможной модификацией заголовков)
`pktmod`	Модифицировать заголовки пакета (fooling)

HTTP модификации:

Функция	Описание
`http_domcase`	Изменить регистр домена (HoSt)
`http_hostcase`	Изменить регистр заголовка Host
`http_methodeol`	Модифицировать конец строки метода

TCP сплит и disorder:

Функция	Описание
`multisplit`	Разбить пакет на несколько TCP сегментов
`multidisorder`	Разбить + отправить в обратном порядке
`tcpseg`	TCP сегментация

Fake-атаки:

Функция	Описание
`fake`	Отправить fake пакет
`fakedsplit`	Fake + сплит оригинала
`fakeddisorder`	Fake + disorder оригинала
`hostfakesplit`	Fake только для хоста + сплит

SYN-атаки:

Функция	Описание
`syndata`	Отправить SYN с данными
`synack`	Работа с SYN/ACK
`synack_split`	Сплит по SYN/ACK

Window size:

Функция	Описание
`wsize`	Изменить window size на SYN-ACK
`wssize`	Изменить window size на всех пакетах

Прочее:

Функция	Описание
`rst`	Отправить RST
`udplen`	Изменить длину UDP
`dht_dn`	DHT domain name injection

Из `zapret-lib.lua`:

Функция	Описание
`pass`	Ничего не делать (для отладки)
`pktdebug`	Вывести содержимое desync в лог
`argdebug`	Вывести аргументы в лог
`posdebug`	Вывести позиции conntrack
`luaexec`	Выполнить произвольный Lua код

💡 Примеры использования

Простой fake:

```
--lua-desync=fake:blob=fake_default_tls
```

Fake с параметрами:

```
--lua-desync=fake:blob=fake_default_tls:tcp_md5:ip_ttl=3:repeats=5
```

Multisplit:

```
--lua-desync=multisplit:pos=1,midsld
```

Комбинация функций:

```
--lua-desync=fake:blob=fake_default_tls:tcp_md5 \
--lua-desync=multisplit:pos=1,midsld
```

С фильтром payload:

```
--payload=tls_client_hello --lua-desync=fake:blob=fake_default_tls \
--payload=http_req --lua-desync=fake:blob=fake_default_http
```

📊 Структура параметров

```
--lua-desync=функция:param1=val1:param2=val2:param3
             ──────────┬──────────────────────────────
                       │
                       │
          параметры через двоеточие
```

Типы параметров:

- `param=value` — параметр со значением
- `param` — булевый параметр (без значения = true)

🔍 Что получает функция

Каждая функция получает таблицу `desync` с данными:

```lua
desync = {
    -- Аргументы функции
    arg = { blob = "...", tcp_md5 = true, ... },

    -- Информация о функции
    func = "fake",
    func_n = 1,
    func_instance = "1:fake:1",

    -- Диссект пакета
    dis = {
        ip = { ... },       -- или ip6 = { ... }
        tcp = { ... },      -- или udp = { ... }
        payload = "...",
    },

    -- Направление
    outgoing = true/false,

    -- Тип payload
    l7payload = "tls_client_hello",

    -- Conntrack (если есть)
    track = {
        lua_state = {},     -- для хранения данных между пакетами
        tcp = { seq = ..., ack = ... },
        ...
    },

    -- Reassembled данные (если есть)
    reasm_data = "...",

    -- Другие поля...
}
```

🎯 Полный пример

```cmd
winws2 ^
  --wf-tcp-out=80,443 ^
  --lua-init=@zapret-lib.lua --lua-init=@zapret-antidpi.lua ^
  --filter-tcp=80 --filter-l7=http ^
  --payload=http_req --lua-desync=fake:blob=fake_default_http:tcp_md5 ^
  --lua-desync=multisplit:pos=method+2 ^
  --new ^
  --filter-tcp=443 --filter-l7=tls ^
  --payload=tls_client_hello --lua-
desync=fake:blob=fake_default_tls:tcp_md5:tls_mod=rnd,rndsni ^
  --lua-desync=multidisorder:pos=1,midsld
```

✅ Итог

`--lua-desync` — это сердце nfqws2:

- 🔷 Вызывает Lua функцию для каждого пакета
- 🔷 Передаёт параметры через двоеточие
- 🔷 Можно указывать несколько раз (выполняются последовательно)
- 🔷 Работает с фильтрами `--payload`, `--out-range`, `--in-range`
- 🔷 Можно писать свои функции

📋 Полная таблица функций `--lua-desync`

🔷 Базовые функции (zapret-lib.lua)

Функция	Описание	Параметры
`pass`	Ничего не делает (для отладки)	—
`pktdebug`	Выводит содержимое desync в лог	—
`argdebug`	Выводит аргументы функции в лог	—
`posdebug`	Выводит позиции conntrack в лог	—
`luaexec`	Выполняет произвольный Lua код	`code=<lua_code>`

🔷 Функции из zapret-antidpi.lua

Базовые действия

Функция	Аналог nfqws1	Описание	Параметры
`drop`	—	Отбросить пакет	`dir`, `payload`
`send`	`--dup`	Отправить копию пакета	`dir`, fooling, `ip_id`, `ipfrag`, `rawsend`, `reconstruct`
`pktmod`	`--orig`	Модифицировать текущий пакет	`dir`, fooling, `ip_id`

HTTP модификации

Функция	Аналог nfqws1	Описание	Параметры
`http_domcase`	`--domcase`	Изменить регистр домена (HoSt)	`dir`
`http_hostcase`	`--hostcase`	Изменить регистр заголовка Host	`dir`, `spell=<str>` (4 символа)
`http_methodeol`	`--methodeol`	Модифицировать EOL метода	`dir`, `method=cr|lf|crlf|lfcr`, `no_space`

SYN-атаки

Функция	Аналог nfqws1	Описание	Параметры
`syndata`	`--dpi-desync=syndata`	Отправить SYN с данными	`blob=<blob>`, `tls_mod=<list>`, fooling, `rawsend`, `reconstruct`, `ipfrag`
`synack`	—	Отправить SYN-ACK	fooling, `rawsend`, `reconstruct`, `ipfrag`
`synack_split`	—	Сплит по SYN-ACK	`pos=<posmarker>`, `seqovl=N`, `seqovl_pattern=<blob>`

Window size

Функция	Аналог nfqws1	Описание	Параметры
`wsize`	`--wssize`	Изменить window size на SYN-ACK	`wsize=N`, `scale=N`
`wssize`	`--wssize`	Изменить window size на всех пакетах	`wsize=N`, `scale=N`

RST

Функция	Аналог nfqws1	Описание	Параметры
`rst`	`--dpi-desync=rst`	Отправить RST	`dir`, `payload`, fooling, `ip_id`, `rawsend`, `reconstruct`, `ipfrag`, `rstack`

Fake-атаки

Функция	Аналог nfqws1	Описание	Параметры
`fake`	`--dpi-desync=fake`	Отправить fake пакет	`blob=<blob>` *(обязательный)*, `tls_mod=<list>`, `dir`, `payload`, fooling, `ip_id`, `rawsend`, `reconstruct`, `ipfrag`

Сплит и disorder

Функция	Аналог nfqws1	Описание	Параметры
`multisplit`	`--dpi-desync=multisplit`	Разбить на TCP сегменты	`pos=<posmarker_list>`, `seqovl=N`, `seqovl_pattern=<blob>`, `blob=<blob>`, `nodrop`
`multidisorder`	`--dpi-desync=multidisorder`	Разбить + обратный порядок	`pos=<posmarker_list>`, `seqovl=<posmarker>`, `seqovl_pattern=<blob>`, `blob=<blob>`, `nodrop`
`tcpseg`	—	TCP сегментация по диапазону	`pos=<range>` *(обязательный)*, `seqovl=N`, `seqovl_pattern=<blob>`, `blob=<blob>`

Fake + сплит комбинации

Функция	Аналог nfqws1	Описание	Параметры
`hostfakesplit`	`--dpi-desync=hostfakesplit`	Fake только для хоста + сплит	`host=<template>`, `midhost=<posmarker>`, `nofake1`, `nofake2`, `disorder_after=<posmarker>`, `blob=<blob>`, `nodrop`
`fakedsplit`	`--dpi-desync=fakedsplit`	Fake + сплит оригинала	`pos=<posmarker>`, `nofake1`, `nofake2`, `nofake3`, `nofake4`, `pattern=<blob>`, `seqovl=N`, `seqovl_pattern=<blob>`, `blob=<blob>`, `nodrop`
`fakeddisorder`	`--dpi-desync=fakeddisorder`	Fake + disorder оригинала	`pos=<posmarker>`, `nofake1`, `nofake2`, `nofake3`, `nofake4`, `pattern=<blob>`, `seqovl=<posmarker>`, `seqovl_pattern=<blob>`, `blob=<blob>`, `nodrop`

UDP

Функция	Аналог nfqws1	Описание	Параметры
`udplen`	`--dpi-desync=udplen`	Изменить длину UDP пакета	`dir`, `payload`, `min=N`, `max=N`, `increment=N` (по умолч. 2), `pattern=<blob>`, `pattern_offset=N`
`dht_dn`	`--dpi-desync=tamper` (dht)	DHT domain name injection	`dir`, `dn=N` (по умолч. 2)

🔧 Стандартные параметры (применимы ко многим функциям)

Direction (направление)

Параметр	Описание
`dir=in`	Только входящие пакеты
`dir=out`	Только исходящие (по умолчанию)
`dir=any`	Оба направления

Payload фильтр

Параметр	Описание
`payload=<type_list>`	Фильтр по типу payload

Fooling (обманки)

Параметр	Описание
`ip_ttl=N`	Установить TTL IPv4
`ip6_ttl=N`	Установить Hop Limit IPv6
`ip_autottl=delta,min-max`	Автоматический TTL
`ip6_autottl=delta,min-max`	Автоматический Hop Limit
`ip6_hopbyhop[=hex]`	Добавить Hop-by-Hop заголовок
`ip6_hopbyhop2[=hex]`	Второй Hop-by-Hop
`ip6_destopt[=hex]`	Добавить Destination Options
`ip6_destopt2[=hex]`	Второй Destination Options
`ip6_routing[=hex]`	Добавить Routing заголовок
`ip6_ah[=hex]`	Добавить Authentication Header
`tcp_seq=N`	Добавить к TCP sequence
`tcp_ack=N`	Добавить к TCP ack
`tcp_ts=N`	Добавить к timestamp
`tcp_md5[=hex]`	Добавить TCP MD5 опцию
`tcp_flags_set=<list>`	Установить TCP флаги
`tcp_flags_unset=<list>`	Снять TCP флаги
`tcp_ts_up`	Переместить timestamp наверх
`fool=<function>`	Пользовательская функция обманки

Reconstruct

Параметр	Описание
`badsum`	Невалидная контрольная сумма L4

Rawsend

Параметр	Описание
`repeats=N`	Количество повторов
`ifout=<iface>`	Интерфейс отправки
`fwmark=N`	fwmark пакета

IP ID

Параметр	Описание
`ip_id=seq\\|rnd\\|zero\\|none`	Политика IP ID
`ip_id_conn`	Сохранять IP ID между пакетами

IP фрагментация

Параметр	Описание
`ipfrag[=function]`	Включить IP фрагментацию
`ipfrag_disorder`	Отправить фрагменты в обратном порядке
`ipfrag_pos_tcp=N`	Позиция фрагментации TCP (кратно 8)
`ipfrag_pos_udp=N`	Позиция фрагментации UDP (кратно 8)
`ipfrag_next=N`	Next proto для второго фрагмента

TLS модификации

Параметр	Описание
`tls_mod=rnd`	Рандомизировать поле random
`tls_mod=rndsni`	Случайный SNI
`tls_mod=sni=<domain>`	Установить конкретный SNI
`tls_mod=dupsid`	Копировать Session ID
`tls_mod=padencap`	Padding encapsulation

📍 Position markers (маркеры позиций)

Маркер	Описание
`N`	Абсолютная позиция (число)
`-N`	Позиция с конца
`host`	Начало хоста
`endhost`	Конец хоста
`sld`	Second-level domain
`midsld`	Середина SLD
`endsld`	Конец SLD
`method`	HTTP метод
`extlen`	TLS extensions length
`sniext`	TLS SNI extension
`marker+N`	Маркер + смещение
`marker-N`	Маркер - смещение

💡 Примеры

```
# Простой fake
--lua-desync=fake:blob=fake_default_tls:tcp_md5

# Fake с TTL и повторами
--lua-desync=fake:blob=fake_default_tls:ip_ttl=3:repeats=5:tls_mod=rnd,rndsni

# Multisplit с seqovl
--lua-desync=multisplit:pos=1,midsld:seqovl=10

# Fakedsplit
--lua-desync=fakedsplit:pos=midsld:nofake1:tcp_md5

# Комбинация fake + multidisorder
--lua-desync=fake:blob=fake_default_tls:tcp_md5 --lua-desync=multidisorder:pos=1,midsld

# UDP length
--lua-desync=udplen:increment=4:min=50
```

Соответствие флагов nfqws1 → nfqws2

В nfqws1 были готовые флаги `--dpi-desync-fooling=`. В nfqws2 их нет — вместо этого используются **отдельные параметры** для каждого действия.

📋 Таблица соответствия

nfqws1 флаг	nfqws2 параметр	Описание
`md5sig`	`tcp_md5`	Добавить TCP MD5 signature опцию
`badsum`	`badsum`	Невалидная контрольная сумма L4
`badseq`	`tcp_seq=-10000`	Сдвинуть TCP sequence (для SYN)
`badseq`	`tcp_ack=-66000`	Сдвинуть TCP ack (для данных)
`datanoack`	`tcp_flags_unset=ack`	Снять флаг ACK
`badack`	`tcp_ack=-66000`	Сдвинуть TCP ack
`hopbyhop`	`ip6_hopbyhop`	IPv6 Hop-by-Hop header
`hopbyhop2`	`ip6_hopbyhop2`	Второй Hop-by-Hop header
`destopt`	`ip6_destopt`	IPv6 Destination Options
`destopt2`	`ip6_destopt2`	Второй Destination Options
`ipfrag1`	`ipfrag`	IP фрагментация

🔧 Детали каждого флага

`md5sig` → `tcp_md5`

Добавляет TCP опцию MD5 signature (RFC 2385). DPI не может проверить подпись.

```
# nfqws1
--dpi-desync-fooling=md5sig

# nfqws2
--lua-desync=fake:blob=fake_default_tls:tcp_md5
```

`badsum` → `badsum`

Делает контрольную сумму TCP/UDP невалидной. Сервер отбросит пакет.

```
# nfqws1
--dpi-desync-fooling=badsum

# nfqws2
--lua-desync=fake:blob=fake_default_tls:badsum
```

`badseq` → `tcp_seq` / `tcp_ack`

В nfqws1 `badseq` применял разные значения для SYN и обычных пакетов:

- SYN пакеты: `tcp_seq=-10000`
- Обычные пакеты: `tcp_ack=-66000`

```
# nfqws1
--dpi-desync-fooling=badseq

# nfqws2 (для обычных пакетов)
--lua-desync=fake:blob=fake_default_tls:tcp_ack=-66000

# nfqws2 (для SYN)
--lua-desync=fake:blob=fake_default_tls:tcp_seq=-10000
```

`badack` → `tcp_ack`

Сдвигает TCP acknowledgment number.

```
# nfqws1
--dpi-desync-fooling=badack

# nfqws2
--lua-desync=fake:blob=fake_default_tls:tcp_ack=-66000
```

Важно! Для Linux нужен `tcp_ts_up` чтобы работало без `badseq`:

```
--lua-desync=fake:blob=fake_default_tls:tcp_ack=-66000:tcp_ts_up
```

`datanoack` → `tcp_flags_unset=ack`

Снимает флаг ACK с пакета.

```
# nfqws1
--dpi-desync-fooling=datanoack

# nfqws2
--lua-desync=fake:blob=fake_default_tls:tcp_flags_unset=ack
```

`hopbyhop` → `ip6_hopbyhop`

Добавляет IPv6 Hop-by-Hop extension header.

```
# nfqws1
--dpi-desync-fooling=hopbyhop

# nfqws2
--lua-desync=fake:blob=fake_default_tls:ip6_hopbyhop
```

`hopbyhop2` → `ip6_hopbyhop2`

Добавляет **второй** Hop-by-Hop header (нестандартно, ломает обработку).

```
# nfqws1
--dpi-desync-fooling=hopbyhop2

# nfqws2
--lua-desync=fake:blob=fake_default_tls:ip6_hopbyhop2
```

`destopt` → `ip6_destopt`

Добавляет IPv6 Destination Options header.

```
# nfqws1
--dpi-desync-fooling=destopt

# nfqws2
--lua-desync=fake:blob=fake_default_tls:ip6_destopt
```

`ipfrag1` → `ipfrag`

IP фрагментация пакета.

```
# nfqws1
--dpi-desync-fooling=ipfrag1

# nfqws2
--lua-desync=send:ipfrag
```

💡 Примеры комбинаций

Классический fake с md5sig:

```
# nfqws1
--dpi-desync=fake --dpi-desync-fooling=md5sig

# nfqws2
--lua-desync=fake:blob=fake_default_tls:tcp_md5
```

Fake c badseq + md5sig:

```
# nfqws1
--dpi-desync=fake --dpi-desync-fooling=badseq,md5sig

# nfqws2
--lua-desync=fake:blob=fake_default_tls:tcp_ack=-66000:tcp_md5
```

Fake c TTL:

```
# nfqws1
--dpi-desync=fake --dpi-desync-ttl=3

# nfqws2
--lua-desync=fake:blob=fake_default_tls:ip_ttl=3
```

Fake c autottl:

```
# nfqws1
--dpi-desync=fake --dpi-desync-autottl=-1,3-20

# nfqws2
--lua-desync=fake:blob=fake_default_tls:ip_autottl=-1,3-20:ip6_autottl=-1,3-20
```

Fakedsplit c badseq + md5sig:

```
# nfqws1
--dpi-desync=fakedsplit --dpi-desync-fooling=badseq,md5sig --dpi-desync-split-pos=2

# nfqws2
--lua-desync=fakedsplit:pos=2:tcp_ack=-66000:tcp_md5
```

IPv6 hopbyhop + destopt:

```
# nfqws1
--dpi-desync=fake --dpi-desync-fooling=hopbyhop,destopt

# nfqws2
--lua-desync=fake:blob=fake_default_tls:ip6_hopbyhop:ip6_destopt
```

📝 Полный список fooling параметров nfqws2

Параметр	Значение	Описание
`tcp_md5`	`[=hex]`	TCP MD5 signature (16 байт)
`tcp_seq`	`=N`	Сдвиг TCP sequence
`tcp_ack`	`=N`	Сдвиг TCP ack
`tcp_ts`	`=N`	Сдвиг TCP timestamp
`tcp_flags_set`	`=list`	Установить TCP флаги
`tcp_flags_unset`	`=list`	Снять TCP флаги
`tcp_ts_up`	—	Переместить timestamp наверх
`badsum`	—	Невалидная контрольная сумма
`ip_ttl`	`=N`	Установить TTL
`ip6_ttl`	`=N`	Установить Hop Limit
`ip_autottl`	=delta,min-max	Авто TTL
`ip6_autottl`	=delta,min-max	Авто Hop Limit
`ip6_hopbyhop`	`[=hex]`	Hop-by-Hop header
`ip6_hopbyhop2`	`[=hex]`	Второй Hop-by-Hop
`ip6_destopt`	`[=hex]`	Destination Options
`ip6_destopt2`	`[=hex]`	Второй Destination Options
`ip6_routing`	`[=hex]`	Routing header
`ip6_ah`	`[=hex]`	Authentication header

⚡ Быстрая шпаргалка

```
# md5sig
:tcp_md5

# badsum
:badsum

# badseq (для данных)
:tcp_ack=-66000

# badseq (для SYN)
:tcp_seq=-10000

# badack
:tcp_ack=-66000:tcp_ts_up

# datanoack
:tcp_flags_unset=ack

# TTL
:ip_ttl=3:ip6_ttl=3

# autottl
:ip_autottl=-1,3-20:ip6_autottl=-1,3-20

# IPv6 headers
:ip6_hopbyhop:ip6_destopt
```

Нет, `badseq` и `badack` — это **разные** вещи!

📊 Разница

Флаг	Что меняет	Поле TCP	Типичное значение
`badseq`	Sequence number	`tcp.th_seq`	`-10000`
`badack`	Acknowledgment number	`tcp.th_ack`	`-66000`

🔬 Как работают

`badseq` (tcp_seq)

Сдвигает **номер последовательности** (sequence number) пакета.

```
Оригинальный пакет:  seq=1000, ack=5000
С badseq (-10000):   seq=-9000 (990), ack=5000
```

- Сервер видит пакет с "неправильным" seq
- Пакет **не попадает в окно приёма** → отбрасывается
- DPI может обработать пакет, но сервер его игнорирует

`badack` (tcp_ack)

Сдвигает **номер подтверждения** (acknowledgment number) пакета.

```
Оригинальный пакет:  seq=1000, ack=5000
С badack (-66000):   seq=1000, ack=-61000
```

- Сервер видит пакет с "неправильным" ack
- Пакет может быть отброшен или обработан (зависит от ОС)
- На Linux с опцией timestamp пакет может быть принят!

⚠️ Важный нюанс для Linux

На Linux **badack без badseq** может не работать, если есть TCP timestamp опция!

Linux отбрасывает пакеты с плохим ack только если timestamp опция идёт **первой** в списке TCP опций.

Поэтому в nfqws2 есть параметр `tcp_ts_up` — он перемещает timestamp наверх:

```
# Чтобы badack работал на Linux:
--lua-desync=fake:blob=fake_default_tls:tcp_ack=-66000:tcp_ts_up
```

📝 В nfqws1

В nfqws1 флаг `badseq` был **универсальным** и применял разные значения в зависимости от типа пакета:

Тип пакета	Что менялось	Значение
SYN	`tcp_seq`	`-10000`
Данные	`tcp_ack`	`-66000`

Поэтому `--dpi-desync-fooling=badseq` фактически менял **либо seq, либо ack** в зависимости от контекста.

🎯 В nfqws2

В nfqws2 нет такой "магии" — вы **явно указываете** что хотите изменить:

```
# Изменить sequence (для SYN)
:tcp_seq=-10000

# Изменить ack (для данных)
:tcp_ack=-66000

# Изменить оба
:tcp_seq=-10000:tcp_ack=-66000
```

💡 Рекомендации

Для fake пакетов (данные):

```
# Вариант 1: badack с tcp_ts_up (надёжнее на Linux)
--lua-desync=fake:blob=fake_default_tls:tcp_ack=-66000:tcp_ts_up

# Вариант 2: md5sig (проще, работает везде)
--lua-desync=fake:blob=fake_default_tls:tcp_md5

# Вариант 3: TTL (если известно расстояние до DPI)
--lua-desync=fake:blob=fake_default_tls:ip_ttl=3
```

Для syndata (SYN пакеты):

```
--lua-desync=syndata:blob=fake_default_tls:tcp_seq=-10000
```

✅ Итог

	`badseq`	`badack`
Поле	Sequence	Acknowledgment
nfqws2 параметр	`tcp_seq=N`	`tcp_ack=N`
Типичное значение	`-10000`	`-66000`
Для SYN	✅ Да	❌ Нет
Для данных	⚠️ Редко	✅ Да
Нужен tcp_ts_up	Нет	Да (на Linux)

Короткий ответ: `badseq` меняет **seq**, `badack` меняет **ack** — это разные поля TCP заголовка.

`tcp_ts_up` — нужен **не всегда**, но рекомендуется для Linux

🔍 Когда `tcp_ts_up` НУЖЕН

Только на Linux, и только если:

1. Вы используете `tcp_ack` (badack) **без** `tcp_seq` (badseq)
2. В пакете **есть** TCP timestamp опция

Почему?

Linux проверяет timestamp опцию для валидации пакетов. Но есть баг/особенность:

- Linux отбрасывает пакеты с плохим ack **только если timestamp идёт первой** в списке TCP опций
- Если timestamp не первая — пакет может быть **принят** несмотря на плохой ack!

`tcp_ts_up` перемещает timestamp в начало списка опций → Linux корректно отбрасывает пакет.

✅ Когда `tcp_ts_up` НЕ нужен

1. **Целевой сервер — не Linux** (Windows, BSD и др.)
2. **Используете** `tcp_seq` (badseq) — тогда пакет отбрасывается по другой причине
3. **Используете** `tcp_md5` — md5sig работает независимо
4. **Используете** `ip_ttl` — пакет не дойдёт до сервера
5. **В пакете нет timestamp опции** — нечего перемещать

📊 Таблица: когда нужен `tcp_ts_up`

Fooling	Linux	Windows/BSD	`tcp_ts_up` нужен?
`tcp_ack` только	⚠️ Может не работать	✅ Работает	**Да** (для Linux)
`tcp_seq` только	✅ Работает	✅ Работает	Нет
`tcp_ack` + `tcp_seq`	✅ Работает	✅ Работает	Нет
`tcp_md5`	✅ Работает	✅ Работает	Нет
`ip_ttl`	✅ Работает	✅ Работает	Нет

💡 Рекомендация

Если не уверены — добавляйте `tcp_ts_up`. Он не навредит, но гарантирует работу на Linux:

```
# Безопасный вариант (работает везде)
--lua-desync=fake:blob=fake_default_tls:tcp_ack=-66000:tcp_ts_up

# Или просто используйте md5sig — проще и надёжнее
--lua-desync=fake:blob=fake_default_tls:tcp_md5
```

🎯 Короткий ответ

Нет, не обязательно, но:

- На **Linux** без `tcp_ts_up` badack может **не сработать**
- На **Windows/BSD** работает и без него
- **Рекомендуется** добавлять для универсальности

`autottl` — формат **обязателен**, значений по умолчанию нет!

❌ Нельзя использовать без цифр

```
# ОШИБКА! Так нельзя:
--lua-desync=fake:blob=fake_default_tls:ip_autottl

# ОШИБКА! Так тоже нельзя:
--lua-desync=fake:blob=fake_default_tls:ip_autottl=
```

Будет ошибка:

```
parse_autottl: invalid value '...'
```

✅ Обязательный формат

```
ip_autottl=delta,min-max
ip6_autottl=delta,min-max
```

Параметр	Описание
`delta`	Сдвиг от вычисленного hop count (может быть отрицательным)
`min`	Минимальное значение TTL
`max`	Максимальное значение TTL

💡 Примеры

```
# Стандартный вариант: delta=-1, диапазон 3-20
:ip_autottl=-1,3-20:ip6_autottl=-1,3-20

# Более агрессивный: delta=-2
:ip_autottl=-2,3-20:ip6_autottl=-2,3-20

# Осторожный: delta=0 (точно до DPI)
:ip_autottl=0,3-20:ip6_autottl=0,3-20
```

🔬 Как работает autottl

1. **Получает incoming TTL** из ответа сервера (SYN-ACK)
2. **Угадывает начальный TTL** сервера (64, 128 или 255)
3. **Вычисляет hop count** = начальный TTL - incoming TTL
4. **Применяет формулу**: fake_ttl = hop_count + delta
5. **Ограничивает** результат диапазоном min-max

Пример расчёта:

```
Incoming TTL = 52
Угаданный начальный TTL = 64
Hop count = 64 - 52 = 12
delta = -1
Результат = 12 + (-1) = 11

Если min=3, max=20: итоговый TTL = 11 ✓
```

⚠️ Важно

- **Требует conntrack** — нужно видеть входящие пакеты для определения TTL
- **Не работает** если incoming TTL не попадает в диапазоны (32-64, 96-128, 223-255)
- **Fallback на ip_ttl** — если autottl не сработал, используется `ip_ttl` если указан

```
# С fallback на фиксированный TTL:
:ip_ttl=5:ip_autottl=-1,3-20
```

📝 Типичное использование

```
# Полный пример с autottl и fallback
--lua-desync=fake:blob=fake_default_tls:tcp_md5:ip_ttl=5:ip_autottl=-1,3-
20:ip6_ttl=5:ip6_autottl=-1,3-20
```